

Binary Relevance on Distributed Setting for Very Wide Tabular Datasets

MILESTONE 2: Parallel Execution of Binary Classifiers

Mustafa (28426)

Fatima (29294)

Sohaib (28994)

Sameem (29209)

Contents

1	Project Overview	3
1.1	Binary Relevance	3
1.2	Recap of Milestone 1	3
2	Milestone 2	3
2.1	Output Label Preparation	3
2.2	Binary Classifier Design & Small-Scale Validation	4
2.3	Node-Level Training Pipeline	4
2.4	Task Distribution, Monitoring & Failure Handling	5
2.5	Model Collection & Aggregation Verification	5
3	Artefacts Generated	6
4	Remaining Work	7
4.1	Milestone 3 — Aggregation and Final Prediction	7
4.2	Future Infrastructure Note: Transition to Multi-Machine Cluster	8

1 Project Overview

This project implements Binary Relevance (BR), a multi-label classification technique, in a distributed computing environment designed to handle very wide tabular datasets. The dataset used is the EURLex dataset, containing 15,377 training samples, 3,971 test samples, and 5,000 feature columns — a classic example of a very wide, high-dimensional, sparse dataset.

1.1 Binary Relevance

Binary Relevance decomposes a multi-label problem with L labels into L independent binary classification problems. One classifier is trained per label, and final predictions are obtained by combining all classifiers' outputs. The key structural property — full independence between classifiers — makes BR naturally suited to distributed systems, enabling parallel training with zero synchronisation overhead.

1.2 Recap of Milestone 1

Milestone 1 established the full data pipeline: raw CSV ingestion and schema validation, Min-Max normalisation across 5,000 feature columns (computed exclusively from training data), and vertical column-based sharding into 4 partition files. Each partition holds all 15,377 rows but only 1,250 features, along with `row_id` and `labels` columns, enabling each node to train independently. A `Dask LocalCluster` with 4 workers was configured and validated, and all partition metadata was persisted to `partition_metadata.pkl` for handoff to Milestone 2.

2 Milestone 2

Milestone 2 covers the full pipeline from label preparation through parallel distributed classifier training, progress monitoring, failure handling, and trained model collection and verification. All tasks were completed successfully. The notebook (`milestone2.ipynb`) was structured and executed end-to-end with all cells producing correct outputs.

2.1 Output Label Preparation

Label parsing and full binary target matrix construction were completed.

- Parsed the space-separated `labels` column across all 15,377 training rows using a robust `parse_labels` function, handling NaN values, empty strings, and colon-separated index:value formats.
- Computed label frequency across the entire dataset: identified all unique output labels, their occurrence counts, the most and least frequent labels, and the full ID range.
- Constructed `Y_small`: a binary target matrix for the top 12 most frequent labels, used exclusively for small-scale classifier validation.
- Constructed `Y_full`: a complete binary target matrix covering all unique labels in the dataset, used for full distributed training. Each column represents one label as

a binary vector over all training samples.

- Produced a two-panel visualisation of the output label distribution across the full dataset: a histogram of label frequencies (with mean and median markers) and a bar chart of the top 30 most frequent labels, saved to `output_label_distribution.png`.

2.2 Binary Classifier Design & Small-Scale Validation

Classifier architecture definition and baseline validation tasks were completed.

- Defined a reusable `BinaryClassifier` class wrapping scikit-learn's `LogisticRegression` with the `liblinear` solver, configurable regularisation strength C , maximum iterations, and random state.
- Handled class imbalance by setting `class_weight='balanced'`, which automatically adjusts per-class weights inversely proportional to class frequencies — critical for the heavily skewed label distributions present in EURLex.
- Split the feature matrix and `Y_small` (top 12 labels) into 80/20 train/validation sets using a fixed random seed for reproducibility.
- Trained one `BinaryClassifier` per label, skipping any label whose training split contained only a single class.
- Evaluated each trained model on the validation split, reporting per-label accuracy, precision, recall, F1-score, and log-loss, as well as macro-averaged metrics across all trained labels.

2.3 Node-Level Training Pipeline

Feature-label separation, node-level training loop, metric tracking, and model persistence tasks were completed.

- Reconstructed the complete 5,000-feature input matrix from the 4 vertical partition files produced in Milestone 1, verifying row-order consistency across all shards before concatenation.
- Performed feature-label separation for distributed training: split `X_full` and `Y_full` into 80/20 train/validation sets using the same random seed as the small-scale validation step.
- Assigned all unique label columns from `Y_full` to 4 worker nodes using `numpy.array_split`, producing evenly balanced label batches with approximately 989 labels per node.
- Implemented the `train_label_batch_worker` function, executed entirely within each worker process, which:
 - Iterates over its assigned label columns.
 - Skips labels with single-class training splits and logs them as `skipped_single_class_train`.
 - Trains a `LogisticRegression` model per label with class balancing enabled and records wall-clock fit time.
 - Computes train and validation accuracy and log-loss for each trained label.

- Saves each trained model as a `.pkl` file and additionally persists model coefficients and intercepts as `.npz` arrays for lightweight downstream access.
- Writes a per-node `node_metrics.csv` file recording all label-level results.
- Returns a node summary dictionary and a metrics DataFrame to the scheduler.

2.4 Task Distribution, Monitoring & Failure Handling

Task assignment, parallel dispatch, progress monitoring, and retry logic tasks were completed.

- Initialised a Dask `LocalCluster` with 4 workers (1 thread each, processes mode) and a `Client` connected to it, simulating a 4-node distributed environment.¹
- Dispatched 4 non-blocking training tasks to workers via `client.submit()`, each carrying its assigned label batch, the full train/validation feature splits, and the corresponding label target splits.
- Monitored task completion in real time using `as_completed()`, printing per-node status upon each worker’s return: node ID, trained label count, skipped label count, feature count, and worker PID.
- After all tasks completed, re-inspected all Dask future objects for `status == ‘error’`; any failed futures were added to a retry queue.
- Implemented a retry loop with a maximum of 2 retries per failed node: each retry re-submits the same worker function via `client.submit()` and attempts to collect the result; persistent failures are logged as permanently failed.
- Produced a node-level progress summary table recording assigned labels, trained labels, skipped labels, features used, worker PID, and attempt status (`success_first_attempt`, `retried_Nx`, or `permanently_failed`) for every node.
- Closed the Dask client and cluster cleanly after all retry and monitoring work was complete.

2.5 Model Collection & Aggregation Verification

Trained model collection, serialisation format, integrity verification, and complete aggregation verification tasks were completed.

- Collected all trained model entries from the consolidated `all_node_metrics_df`, filtering to `status == ‘trained’` rows only.
- Verified file integrity for each collected model by checking that all three associated files exist — `.pkl` model file, `_coef.npz` coefficient array, and `_intercept.npz`

¹Originally designed for multi-machine distribution across team members’ laptops over a local network. Due to physical constraints preventing co-location, the implementation was pivoted to a `LocalCluster` simulation. The architecture, partitioning strategy, worker code, and task dispatch logic are identical to a real multi-machine deployment; only the cluster initialisation differs. To deploy on real machines, replace `LocalCluster` with `Client(‘<scheduler-ip>:8786’)` after running `dask scheduler` and `dask worker` on the respective machines.

intercept array — and computing an MD5 checksum of the model file for transfer integrity confirmation.

- Built a `model_registry.csv` file recording, for every trained label: node ID, label name, all three file paths, per-file existence flags, integrity status (OK or MISSING), MD5 checksum, and train/validation accuracy.
- Verified complete model aggregation by comparing the number of `integrity == OK` registry entries against the total trained label count reported in the node training summary, computing a coverage percentage and flagging any discrepancy.
- Saved the `node_training_summary.csv`, `all_node_metrics.csv`, `training_manifest.json`, and `model_registry.csv` files for handoff to Milestone 3.

3 Artefacts Generated

Successful execution of `milestone2.ipynb` produces the following output files:

Table 1: Generated Output Files

File	Description
<code>output_label_distribution.png</code>	Two-panel plot: histogram of all output label frequencies with mean/median markers, and bar chart of the top 30 most frequent labels.
<code>models_node_level/ node_X/</code>	Per-node directory containing one <code>.pkl</code> model file, one <code>_coef.npy</code> , and one <code>_intercept.npy</code> per trained label.
<code>models_node_level/ node_X/node_metrics.csv</code>	Per-node CSV recording label, status, train/val accuracy, train/val log-loss, fit time, features used, and worker PID for every assigned label.
<code>models_node_level/ node_training_summary.csv</code>	One row per node: assigned, trained, and skipped label counts, features used, worker PID, and node metrics path.
<code>models_node_level/ all_node_metrics.csv</code>	Consolidated label-level metrics from all nodes in a single CSV.
<code>models_node_level/ training_manifest.json</code>	JSON document recording cluster configuration, total labels assigned/trained/skipped, and per-node summaries.
<code>models_node_level/ model_registry.csv</code>	Registry of all collected models with file paths, existence flags, MD5 checksums, integrity status, and train/val accuracy per label.

4 Remaining Work

4.1 Milestone 3 — Aggregation and Final Prediction

The third milestone focuses on distributed inference, prediction aggregation, and overall system evaluation.

A distributed prediction pipeline will be implemented where input samples are broadcast to all nodes, and each node generates predictions using its locally stored models. These per-node predictions will then be collected and combined into a unified output across all

labels.

An aggregation strategy will be developed to merge predictions across labels, including thresholding or voting mechanisms, probability aggregation, and handling of conflicting or overlapping outputs. The final predictions will be formatted into a consistent and usable structure suitable for downstream evaluation.

Performance benchmarking will be conducted to evaluate the efficiency of the distributed system. This includes comparing sequential and parallel execution times, calculating speedup and parallel efficiency, testing scalability, and performing iso-efficiency analysis.

Finally, all components from previous milestones will be integrated into a complete end-to-end pipeline. A demonstration workflow will be developed, along with visualisations and supporting materials for presenting the final system.

4.2 Future Infrastructure Note: Transition to Multi-Machine Cluster

The current implementation uses a Dask `LocalCluster`, which simulates a 4-node distributed environment on a single machine. This was a necessary adaptation due to physical constraints preventing team co-location, but the true potential of the distributed design will be realised when deployed on a real multi-machine cluster.

A future upgrade will require: a dedicated scheduler process on one machine with workers on separate machines; network reachability across all nodes (same LAN, VPN, or equivalent); firewall and port configuration for scheduler-worker communication; and shared or pre-distributed data paths across machines.

The work completed across Milestones 1 and 2 — with its clean metadata-driven design, partition handoff structure, and worker functions that are already fully self-contained — is structured to facilitate this upgrade with minimal changes to the core logic.